

Moving a wealth CRM into HubSpot without losing the relationships

A registered investment advisor left Redtail for HubSpot. The hard part was never the contacts — it was householding, the account book, and the audit trail. This is the data-model exercise that decides whether the migration works.

[Paul Maxwell](#), CEO, RevOps HQ

July 2026

ABSTRACT

A mid-market RIA with an affiliated tax practice ran a decade of client history in Redtail — roughly 11,000 contacts, 3,300 investment accounts, 3,300 households, 9,000 relationships, and 180,000 notes — and wanted to standardize on HubSpot. The migration is treated as a data-modeling problem first and a software problem second: inventory the source model, map each entity to a HubSpot object, and find what has no native home. That last step — investment accounts and households — is what forces HubSpot Enterprise, and it is the decision most teams get wrong. This is a first-principles walkthrough of the object model, the association graph, the property parsing, and the idempotent migration that let a compliance-heavy firm move without losing the thing that makes it a wealth firm: the web of relationships.

1 A CRM IS A DATA MODEL BEFORE IT IS SOFTWARE

The instinct on a CRM migration is to export contacts, import contacts, and call the rest configuration. For a transactional sales team that mostly works. For a wealth-management firm it fails on day one, because the unit of value is not the contact. It is the *household*, the *account*, and the *relationships* between people — and none of those are contacts.

So the migration starts as an inventory, not an export. Every distinct kind of thing in the source system is written down and asked one question: what HubSpot object represents this? The answers fall into three buckets — has a native home, needs modeling, or has no home at all — and the third bucket is the whole engagement.

THE FIRST PRINCIPLE

Map the source model before you touch the data. A CRM is a set of objects, the properties on them, and the associations between them. Migrate the model, and the records follow. Migrate the records first, and you spend the next year rebuilding a model by hand under a live team.

2 WHERE A WEALTH CRM AND HUBSPOT DIVERGE

Redtail, like most vertical CRMs, keeps everyone in a single “contact” table and separates them with a type flag: individuals, businesses, trusts, and associations all live together. HubSpot does not work that way. People are Contacts; organizations are Companies; and a trust is neither cleanly. The migration has to *shard one table into several objects* and preserve an Entity-Type property so nothing is lost in the split.

Two source entities have no native HubSpot object at all, and they are the two that matter most:

- **The investment account.** ~3,300 of them, each with balances, ownership roles, custodian, and registration. HubSpot has no standard object for it.
- **The household.** ~3,300 of them — the reporting and billing unit an advisor actually thinks in. Also no standard object.

You can model accounts as Deals and households as Companies on a lower tier, and it looks fine in a demo. It breaks the moment you need labeled relationships. Which brings us to the part that decides the tier.

3 THE ASSOCIATION GRAPH IS THE FIRM

The 9,000 relationships in the source are not decoration; they *are* the business. Spouse to spouse. Parent to child. Trustee, grantor, and beneficiary to a trust. Owner, joint owner, and beneficiary to an account. Head of household to member. Attorney and CPA to a client. Fifty-plus distinct relationship types, many of them on the same pair of records at once.

HubSpot can represent this — with **custom association labels** and **multiple labeled associations per record pair**. Both are Enterprise features. On Professional they import unlabeled, every relationship flattens to a generic link, and the graph that makes the firm legible collapses into noise. This is the single most consequential scoping decision on the project, and it is invisible in a product demo.

THE ENTERPRISE LINE

Custom objects (Account, Household) and labeled multi-associations are the features that separate a real wealth CRM from a toy. If a proposal quotes Professional for a householded book, it has mispriced the job. The tier is a data-model consequence, not a budget preference.

4 EVERYTHING IS A KEYWORD UNTIL YOU MAKE IT A FIELD

Because Redtail lacked dedicated fields for many attributes, the firm did what every team does: it crammed them into keyword strings and user-defined fields. Client tier, do-not-solicit flags, plan types, custodians, prospect stage — all encoded as text. Migrating them verbatim would carry the mess across; the value is in *parsing them apart* into typed HubSpot properties that workflows and reports can actually use.

The build defined roughly 200 properties, grouped by business process — tax, billing, compliance, retirement plans, communication — so a record stays navigable instead of becoming a wall of fields. Keyword clusters became single- and multi-select properties: Tier, Plan Types, Custodian, Prospect Stage, Do-Not-Solicit. Fee blobs split into an amount and a date. Legacy duplicate fields were archived rather than deleted, so nothing was destroyed on the way in.

5 COMPLIANCE IS A WORKFLOW, NOT A CHECKBOX

An RIA has supervisory obligations a flat activity log cannot satisfy: approvals, thresholds, segregation of duties, a record of who signed off. Redtail's free-text activities have none of this, so it was rebuilt as controlled process in HubSpot — ticket pipelines with stage requirements and conditional routing.

The clearest example is money movement. A distribution above a set threshold to a third party cannot simply advance; the workflow routes it to a principal for approval and blocks the stage until that approval exists. The control is enforced by the system, not remembered by a person. In total the firm's 69 legacy activity templates were rationalized to 33 real processes, expressed as 10 ticket pipelines, 22 workflows, seven guided playbooks, and four dashboards.

6 A MIGRATION YOU CAN RUN TWICE

At this volume the migration has to be *idempotent* — safe to run again without doubling anything — because you will run it again. The mechanism is simple and worth stealing: give every record a unique external-ID property carrying its Redtail identifier, and match on it. A note stamped with its source note ID cannot be imported twice. An activity stamped with its source activity ID maps to a single HubSpot task no matter how many times the load runs.

The standard Redtail export quietly dropped columns, so extraction was rebuilt against the raw API with the Redtail ID as the key of record. Every write was logged with counts and a verification step, and a small reconciliation pass compared HubSpot against the source field by field, surfacing a short list of records to restore or recreate — with explicit logic to never recreate a record that already existed. That is how ~180,000 notes and 9,000 labeled associations land once, correctly, and stay that way.

TRANSFERABLE PATTERN

A unique external-ID property per object is the cheapest thing you can build and the one that makes a large migration safe. It turns imports into upserts, makes re-runs free, and gives you a real reconciliation key. Do this before you move a single record.

7 WHAT CHANGED

The firm now runs on a model that matches how it actually works.

Households and accounts are first-class records; a decade of relationships carries their real labels; and the client history — every note, every activity — is intact and searchable in one place instead of split across a legacy tool and a stack of spreadsheets.

The operational effect is that advisors stopped reconstructing context by hand. Preparing for a client meeting had meant cross-referencing Redtail, a portfolio system, and a shared drive; with the household and its accounts, relationships, and open items on one record, that prep collapsed to a single screen and gave each advisor back an estimated four to six hours a week. Intake became a guided playbook instead of tribal knowledge, so a new household is onboarded consistently by anyone on the team. And the money-movement controls mean the compliance story is now something the system proves, not something staff swear they followed.

Figures are representative of the engagement and drawn from RevOps HQ's delivery experience; exact results vary by firm.

8 THE MODEL, GENERALIZED

Strip away the vertical and the exercise is reusable for any migration into HubSpot:

- **Inventory the source model first.** Objects, properties, associations — on paper — before any export.
- **Find what has no native object.** That is where custom objects go, and it usually decides your tier.
- **Treat labeled relationships as data, not garnish.** If association labels matter, that is an Enterprise decision, full stop.
- **Parse encoded fields into typed properties.** Don't carry the keyword soup across.
- **Make it idempotent with external IDs.** Build the re-run safety before the first load.

RELATED STUDIES

The same “model before software” discipline shows up elsewhere: representing a *dealer channel* where the buyer isn't the user, modeling *project-based* revenue when the deal isn't the work, and solving *identity resolution* at an integration boundary where a native sync multiplies records. Different verticals, one exercise.